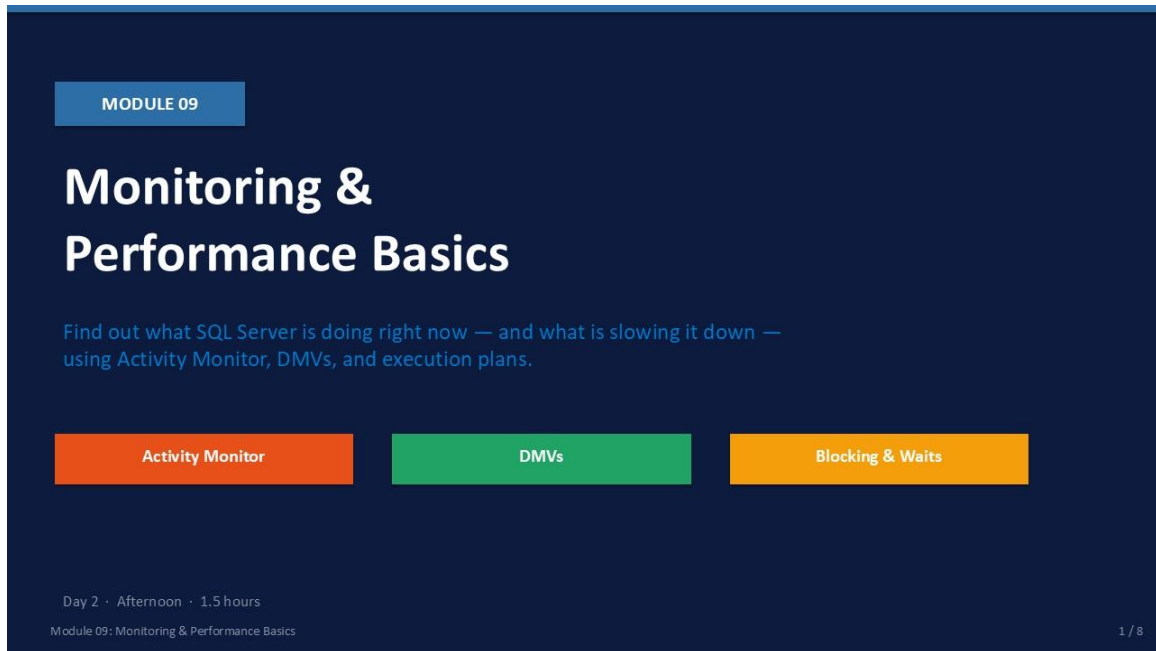


The slide has a dark blue background. At the top left, a light blue box contains the text 'MODULE 09'. Below this, the title 'Monitoring & Performance Basics' is written in large white font. Under the title, a line of smaller white text reads: 'Find out what SQL Server is doing right now — and what is slowing it down — using Activity Monitor, DMVs, and execution plans.' Below this text are three colored rectangular buttons: an orange one labeled 'Activity Monitor', a green one labeled 'DMVs', and a yellow one labeled 'Blocking & Waits'. At the bottom left, small white text indicates 'Day 2 · Afternoon · 1.5 hours' and 'Module 09: Monitoring & Performance Basics'. At the bottom right, '1 / 8' is displayed.

MODULE 09

Monitoring & Performance Basics

Find out what SQL Server is doing right now — and what is slowing it down — using Activity Monitor, DMVs, and execution plans.

Activity Monitor DMVs Blocking & Waits

Day 2 · Afternoon · 1.5 hours
Module 09: Monitoring & Performance Basics

1 / 8

Slide 1 of 8

Module 09: Monitoring & Performance Basics

Module	Day	Duration	Lab
09	2 — Afternoon	90 minutes	Lab 09

When a user reports that "SQL Server is slow," you need a structured method to isolate the problem — not guesswork. This module teaches you the three diagnostic layers every DBA must master: Activity Monitor for a fast visual overview, Dynamic Management Views for precise T-SQL-based investigation, and execution plans to find exactly where a query spends its time.

You will also learn how to identify blocking chains — a very common cause of performance complaints — and how to read wait statistics to understand whether the bottleneck is CPU, I/O, locks, memory, or network. Module 09 ends with a first-response checklist you can apply on the job every time someone says "something is wrong with the database."



Read Before Touching

When investigating a live performance problem,

always observe first. Run read-only DMV queries and check Activity Monitor before making any changes. Killing sessions or restarting services without diagnosis often makes things worse.

What This Module Covers

- **Activity Monitor** CPU, batch requests, waiting tasks, recent expensive queries — all in one GUI.
- **Key DMVs** `sys.dm_exec_requests`, `dm_os_wait_stats`, `dm_exec_sql_text`, and others.
- **Blocking & Deadlocks** Finding the head blocker, understanding deadlock victims, using KILL judiciously.
- **Wait Statistics** Common wait types and what each one tells you about the bottleneck layer.
- **Execution Plans** Reading plans right-to-left, identifying table scans, key lookups, and operator cost.
- **Performance Checklist** Eight-step first-response workflow for unknown performance incidents.

What You Will Learn

By the end of Module 09 you will be able to:

-  Use Activity Monitor to identify active sessions and expensive queries
-  Query key DMVs to see CPU, memory, I/O, and wait statistics
-  Identify a blocking chain and find the head blocker
-  Explain what wait statistics reveal about performance bottlenecks
-  Read a basic query execution plan to find index scan warnings

Module 09: Monitoring & Performance Basics

2 / 8

Slide 2 of 8

Learning Objectives

By the end of Module 09 you will be able to:

- **Use Activity Monitor to identify active sessions and expensive queries** — open Activity Monitor, expand the five panels, and find the session consuming the most CPU.
- **Query key DMVs to see CPU, memory, I/O, and wait statistics** — using `dm_exec_requests`, `dm_os_wait_stats`, and `dm_exec_query_stats`.
- **Identify a blocking chain and find the head blocker** — using the `blocking_session_id` column in `dm_exec_requests`.
- **Explain what wait statistics reveal about performance bottlenecks** — mapping common wait types (`PAGEIOLATCH`, `LCK_M_X`, `CXPACKET`) to their root causes.
- **Read a basic query execution plan to find index scan warnings** — enabling Include Actual Execution Plan, reading right-to-left, spotting Table Scan and Key Lookup.

Activity Monitor

Activity Monitor gives a real-time dashboard of instance health — no T-SQL required.

OVERVIEW

4 live charts: % CPU, Waiting Tasks, Database I/O (MB/s), Batch Requests/sec
Refreshes every 10 seconds by default (configurable).

RESOURCE WAITS

Cumulative wait categories since last reset.
Helps identify whether bottleneck is CPU, I/O, locks, network, or memory.
Higher wait ms = bigger contributor.

PROCESSES

Lists all active SPID sessions.
Columns: Session ID, CPU, Memory, Blocked By, Command, Wait Type, Host.
Right-click a session → Kill Process (use carefully!)

DATA FILE I/O

Per-database file read/write throughput.
Useful for identifying which database is driving I/O.
Check against storage capacity baseline.

RECENT EXPENSIVE QUERIES

Top queries by CPU, elapsed time, logical reads, or physical reads.
Click a row to see the query text and execution plan.
First place to look when 'the server is slow'.

Open: SSMS toolbar → Activity Monitor icon — or — right-click instance → Activity Monitor

Module 09: Monitoring & Performance Basics

3 / 8

Slide 3 of 8

Activity Monitor

Activity Monitor is the fastest way to see what SQL Server is doing right now. It requires no T-SQL and provides a real-time dashboard that auto-refreshes every 10 seconds. Open it from the SSMS toolbar (the bar chart icon) or by right-clicking the instance and selecting Activity Monitor.

The Five Panels

Panel	What It Shows	What to Look For
Overview	4 live charts: CPU %, Waiting Tasks, Database I/O (MB/s), Batch Requests/sec	Sudden CPU spike, sustained waiting tasks > 5
Processes	All active SPID sessions with CPU, memory, blocked-by, and wait type columns	Sessions with a non-zero Blocked By value
Resource Waits	Cumulative wait category totals since last reset	Top wait category drives the next investigation step
Data File I/O	Per-database file read/write throughput in MB/s	One database dominating I/O bandwidth
Recent Expensive Queries	Top queries by CPU, elapsed time, logical reads, or physical reads	Click a row to view query text and execution plan

Killing a Session (Last Resort)

In the Processes panel you can right-click any session and select Kill Process. Use this only when the session is blocking others and cannot be resolved by waiting. A KILL command causes an immediate rollback of the active transaction — this can take a long time for large transactions.



Activity Monitor Has Overhead

Activity Monitor runs queries against the instance every 10 seconds. On a heavily loaded server this can add noticeable overhead. Close Activity Monitor when not actively investigating.

Essential DMVs for Every DBA

Dynamic Management Views surface real-time engine data — far more detail than Activity Monitor.

```
-- Active sessions with wait info
SELECT r.session_id, r.status, r.cpu_time, r.total_elapsed_time,
       r.wait_type, r.wait_time, t.text AS sql_text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50 -- exclude system sessions
ORDER BY r.cpu_time DESC;

-- Top wait types since last restart
SELECT TOP 10 wait_type, wait_time_ms, waiting_tasks_count
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN ('SLEEP_TASK', 'BROKER_TO_FLUSH', 'WAITFOR', ...)
ORDER BY wait_time_ms DESC;

-- Memory usage by database
SELECT database_id, DB_NAME(database_id) AS DBName,
       COUNT(*) * 8 / 1024 AS CacheMB
FROM sys.dm_os_buffer_descriptors
GROUP BY database_id ORDER BY CacheMB DESC;
```

-  **sys.dm_exec_requests**
Currently executing requests — CPU, waits, query text
-  **sys.dm_exec_sessions**
All open sessions — login, host, program name
-  **sys.dm_exec_sql_text**
Table-valued function — returns SQL text from sql_handle
-  **sys.dm_os_wait_stats**
Cumulative wait type counts and durations
-  **sys.dm_os_buffer_descriptors**
Buffer pool pages — which DBs consume most memory
-  **sys.dm_io_virtual_file_stats**
Actual read/write I/O per database file
-  **sys.dm_exec_query_stats**
Cached query plan stats — logical reads, CPU, executions

Module 09: Monitoring & Performance Basics

4 / 8

Slide 4 of 8

Essential DMVs for Every DBA

Dynamic Management Views (DMVs) are system views that surface real-time engine state. They provide far more detail than Activity Monitor and are queryable with T-SQL so results can be filtered, joined, and saved.

DMV	What It Returns
sys.dm_exec_requests	Currently executing requests — CPU time, wait type, blocking session, query handle
sys.dm_exec_sessions	All open sessions — login name, host, application, last request time
sys.dm_exec_sql_text()	Table-valued function — returns SQL text from a sql_handle or plan_handle
sys.dm_os_wait_stats	Cumulative wait type counts and total wait durations since last restart
sys.dm_os_buffer_descriptors	Buffer pool pages — which databases consume the most memory
sys.dm_io_virtual_file_stats()	Read and write I/O counters per database data and log file
sys.dm_exec_query_stats	Aggregated plan cache stats — total CPU, logical reads, execution count

Active Sessions with Wait Info

T-SQL

```
-- Show active requests with their current wait type and SQL text
SELECT
    r.session_id,
    r.status,
    r.cpu_time,
    r.total_elapsed_time,
    r.wait_type,
    r.wait_time,
    t.text AS sql_text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50    -- exclude system sessions
ORDER BY r.cpu_time DESC;
```

Top Wait Types

T-SQL

```
-- Find top 10 wait types since last SQL Server restart
SELECT TOP 10
    wait_type,
    wait_time_ms,
    waiting_tasks_count,
    wait_time_ms / NULLIF(waiting_tasks_count, 0) AS avg_wait_ms
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
    'SLEEP_TASK', 'BROKER_TO_FLUSH', 'WAITFOR',
    'DISPATCHER_QUEUE_SEMAPHORE', 'CLR_AUTO_EVENT'
)
ORDER BY wait_time_ms DESC;
```

Blocking & Deadlocks

Blocking degrades throughput. Deadlocks crash sessions. Both have T-SQL diagnostic queries.

BLOCKING

- Session A holds a lock; Session B waits
- Blocking is normal — only a problem if > 5 sec
- Head blocker: the session that no-one is waiting for
- `sys.dm_exec_requests.blocking_session_id` = head blocker
- Resolution: commit/rollback the long transaction or kill SPID

DEADLOCK

- Session A waits for B; Session B waits for A
- SQL Server detects and kills the 'deadlock victim'
- Victim receives error 1205 — must retry the transaction
- Trace flag 1222 or Extended Events to capture deadlock graph
- Resolution: consistent lock-acquisition order; shorter transactions

FIND THE HEAD BLOCKER

```
SELECT
  r.session_id,
  r.blocking_session_id,
  r.wait_type,
  r.wait_time / 1000.0 AS wait_sec,
  t.text AS blocking_sql
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id > 0;
```

Kill head blocker (last resort):

KILL <session_id>;

```
KILL 58; -- kills session 58
```

Warning: KILL causes an immediate rollback — can be slow for large transactions.

Module 09: Monitoring & Performance Basics

5 / 8

Slide 5 of 8

Blocking & Deadlocks

Blocking and deadlocks are the two most common lock-related performance problems in SQL Server. They look similar but have different causes and require different responses.

Blocking

Blocking occurs when Session A holds a lock on a resource and Session B is waiting for that same resource. Blocking is normal and expected in a transactional database. It only becomes a problem when the wait time exceeds a few seconds or when a chain of blocked sessions grows.

- **Head blocker** The session that no other session is waiting for. Identify it by finding sessions with `blocking_session_id = 0` that other sessions are blocked by.
- **Blocking threshold** The default is 0 seconds (all blocking is recorded). Alert when blocking exceeds 5 seconds.
- **Resolution** Find the head blocker's query, determine why the transaction is open, commit or roll back the transaction. Use KILL only as a last resort.

T-SQL

```
-- Find all blocked sessions and their head blocker
SELECT
  r.session_id,
  r.blocking_session_id,
```



```
r.wait_type,  
r.wait_time / 1000.0    AS wait_sec,  
t.text                  AS blocked_sql  
FROM sys.dm_exec_requests r  
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t  
WHERE r.blocking_session_id > 0;
```

Deadlocks

A deadlock occurs when Session A is waiting for Session B, AND Session B is simultaneously waiting for Session A — a circular wait with no exit. SQL Server automatically detects this, selects a "deadlock victim" (the cheaper transaction to roll back), and terminates it with error 1205.

- **Error 1205** The application receives "Transaction was deadlocked on lock resources with another process and has been chosen as the deadlock victim."
- **Deadlock victim** SQL Server chooses based on SET DEADLOCK_PRIORITY or the transaction with the least rollback cost.
- **Prevention strategy** Access tables in a consistent order across all transactions; keep transactions short; avoid user interaction inside a transaction.
- **Capture deadlocks** Enable trace flag 1222 or use Extended Events with the xml_deadlock_report event to capture deadlock graphs.



KILL Is a Blunt Instrument

KILL <session_id> causes an immediate rollback of the active transaction. For long-running transactions this rollback can take minutes. Always identify the blocking SQL first and try to resolve it gracefully before using KILL.

Wait Statistics & Execution Plans

Wait types tell you WHY queries are slow. Execution plans show you WHERE time is spent.

COMMON WAIT TYPES

	PAGEIOLATCH_SH/_EX	Waiting for a data page to load from disk → I/O bottleneck
	LCK_M_X / LCK_M_S	Lock waits → blocking by another transaction
	CXPACKET / CXCONSUMER	Parallel query coordination → consider MAXDOP tuning
	ASYNC_NETWORK_IO	Client not consuming results fast enough → app layer issue
	SOS_SCHEDULER_YIELD	CPU pressure → queries spinning on scheduler
	WRITELOG	Log flush waits → storage I/O latency on log drive

READING EXECUTION PLANS

Show actual plan:	Query → Include Actual Execution Plan (Ctrl+M)
Read right to left:	Data flows from rightmost operator to leftmost (SELECT)
Thick arrows = cost:	Wider line = more rows estimated — check for skew
Table Scan warning:	Orange exclamation = no suitable index — consider adding one
Key Lookup:	Goes to clustered index for columns not in non-clustered → add INCLUDE
Cost %:	Percentage of total plan cost — highest % = tune first

Estimated vs Actual rows divergence > 10x = statistics are stale → run UPDATE STATISTICS.

Module 09: Monitoring & Performance Basics

6 / 8

Slide 6 of 8

Wait Statistics & Execution Plans

Wait statistics tell you WHY a query is slow. Execution plans tell you WHERE inside the query the time is spent. Together they guide you to the root cause without guesswork.

Common Wait Types Reference

Wait Type	What It Means	Typical Root Cause
PAGEIOLATCH_SH / _EX	Waiting for a data page to load from disk	I/O bottleneck — slow storage or missing index causing large scans
LCK_M_X / LCK_M_S	Lock waits — exclusive or shared	Blocking by another long-running transaction
CXPACKET / CXCONSUMER	Parallel query thread coordination	Consider reducing MAXDOP; one thread lagging behind others
ASYNC_NETWORK_IO	Client is not consuming result rows fast enough	Application-layer issue; large result sets; slow network
SOS_SCHEDULER_YIELD	CPU spinning — waiting for scheduler time	CPU pressure; too many concurrent queries or expensive queries
WRITELOG	Waiting for transaction log flush to complete	Log drive I/O latency; log file on slow storage

When estimated rows diverge from actual rows by more than 10×, statistics are stale. Run UPDATE STATISTICS on the relevant table.

Reading Execution Plans

- **Enable actual plan** In SSMS: Query menu → Include Actual Execution Plan (Ctrl+M). Run the query.
- **Read right to left** Data flows from the rightmost operator (data source) to the leftmost (SELECT). Start your analysis at the right.
- **Thick arrows = rows** The width of connecting arrows represents the estimated number of rows. Unexpectedly wide arrows indicate cardinality issues.
- **Table Scan warning** A yellow exclamation mark on a Table Scan operator means no suitable index was found. Consider adding one.
- **Key Lookup** SQL Server goes back to the clustered index to fetch columns not in the non-clustered index. Add those columns as INCLUDE columns.
- **Operator cost %** Each operator shows its percentage of total plan cost. Tune the highest-cost operator first.

LAB 09

Investigate Dian's Slow Inventory Report

Duration: ~45 min · Tool: SSMS · Database: NovaMart

STEPS

- 1 Open Activity Monitor — observe CPU %, Waiting Tasks, Batch Requests/sec
- 2 Expand 'Recent Expensive Queries' — find the inventory report query
- 3 Run the DMV query for active sessions — review wait_type column
- 4 Run sys.dm_os_wait_stats — identify top 5 wait types
- 5 Open a new query: SELECT * FROM Product ORDER BY ProductName (no index on name)
- 6 Enable 'Include Actual Execution Plan' — run the query
- 7 Read the plan: identify Table Scan warning
- 8 Simulate blocking: open 2 windows, BEGIN TRAN + UPDATE in window 1, SELECT in window 2
- 9 Identify the blocking session using dm_exec_requests
- 10 Roll back window 1 transaction — confirm window 2 unblocks

VALIDATION

- ✓ Activity Monitor loads successfully
- ✓ DMV returns active session rows
- ✓ Top wait types identified
- ✓ Table Scan visible in plan
- ✓ Blocking chain identified via DMV
- ✓ Blocking resolves after ROLLBACK

Module 09: Monitoring & Performance Basics
7 / 8

Slide 7 of 8

Lab 09: Investigate a Slow Inventory Report

Goal	Duration	Tool	Database
Use Activity Monitor, DMVs, execution plans, and blocking simulation to diagnose performance	~45 min	SSMS	NovaMart

Prerequisites

- NovaMart database with Employee and Department tables (from Lab 04).
- SSMS connected to the SQL Server instance.

Lab Steps

1. Open Activity Monitor (right-click the instance → Activity Monitor). Observe the Overview charts.

Expected: Four live charts appear: CPU %, Waiting Tasks, Database I/O, Batch Requests/sec.

2. Expand the Processes panel. Note the columns: Session ID, CPU, Blocked By, Wait Type, Host Name.

Expected: Current sessions are listed. Most should show no wait type or be in background status.

3. Expand Recent Expensive Queries. Click a row to view the query text in the panel at the bottom.

Expected: Query text and execution plan button appear in the lower panel.

4. Open a new query window. Run the active sessions DMV:

```
SELECT r.session_id, r.status, r.cpu_time, r.wait_type, t.text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50
ORDER BY r.cpu_time DESC;
```

Expected: Returns rows for active sessions with wait types visible.

5. Run the top wait types query:

```
SELECT TOP 10 wait_type, wait_time_ms, waiting_tasks_count
FROM sys.dm_os_wait_stats
ORDER BY wait_time_ms DESC;
```

Expected: Returns a list of wait types ordered by total wait time. Note the top 3 wait types.

6. Enable Include Actual Execution Plan (Ctrl+M).

Run: `SELECT * FROM dbo.Employee ORDER BY FirstName;`
(Assumes there is no index on FirstName.)

Expected: Execution plan appears in a new Execution Plan tab.

7. In the execution plan, look for a Table Scan or Clustered Index Scan operator with a yellow warning icon.

Expected: A warning appears indicating a missing index recommendation.

8. Simulate blocking:

Window 1: `BEGIN TRAN; UPDATE dbo.Employee SET DeptID = 1 WHERE EmpID = 1;`
(Do NOT commit yet — leave the transaction open.)

Expected: Transaction is open; row is locked.

9. Window 2: `SELECT * FROM dbo.Employee WHERE EmpID = 1;`
(This should hang — it is blocked by Window 1.)

Expected: Query in Window 2 does not return — it is waiting.

10. Window 3: Run the blocking query:

```
SELECT r.session_id, r.blocking_session_id, r.wait_type, t.text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id > 0;
```

Expected: Blocked session and head blocker session ID are visible.

11. Window 1: `ROLLBACK;`

Observe that Window 2's query now completes.

Expected: Window 2 returns the result row. Blocking resolved.

Validation Checklist

- ☐ Activity Monitor loads and shows all 5 panels
- ☐ DMV query returns active session rows with wait type information
- ☐ Top 5 wait types identified from dm_os_wait_stats
- ☐ Table Scan visible in the execution plan for the Employee SELECT
- ☐ Blocking chain identified in dm_exec_requests (blocking_session_id > 0)
- ☐ Blocking resolves after ROLLBACK in Window 1

Troubleshooting

Symptom	Likely Cause	Resolution
Activity Monitor shows error	Insufficient permissions	Must be sysadmin or have VIEW SERVER STATE permission
DMV returns no rows	No active queries above session 50	Normal in a quiet training environment — run a slow query first
Execution plan tab not showing	Ctrl+M not enabled before running	Enable Include Actual Execution Plan before executing the query
Window 2 does not block	Read committed snapshot isolation on	Check: SELECT is_read_committed_snapshot_on FROM sys.databases WHERE name='NovaMart'

DBA Performance First-Response Checklist

When someone says 'SQL Server is slow', start here.

Check CPU & Batch Requests Activity Monitor overview charts — is CPU spiking or flat?	Check Waiting Tasks Activity Monitor — large Waiting Tasks = blocking or resource pressure
Find Blocking Sessions dm_exec_requests WHERE blocking_session_id > 0	Check Top Wait Types dm_os_wait_stats — top wait category guides next steps
Find Expensive Queries Activity Monitor Recent Expensive Queries OR dm_exec_query_stats	Check Index Fragmentation dm_db_index_physical_stats — run if I/O is the wait type
Check Available Memory dm_os_sys_info.physical_memory_in_use_kb vs total	Review SQL Server Error Log Management → SQL Server Logs — any recent errors or warnings?

Module 09: Monitoring & Performance Basics 8 / 8

Slide 8 of 8

DBA Performance First-Response Checklist

When someone reports "SQL Server is slow," apply this eight-step first-response workflow before making any changes. Each step narrows the problem to the correct layer.

Step	Action	Tool / Query
1	Check CPU & Batch Requests	Activity Monitor overview — is CPU spiking or sustained flat?
2	Check Waiting Tasks	Activity Monitor — high Waiting Tasks = blocking or resource pressure
3	Find Blocking Sessions	SELECT from dm_exec_requests WHERE blocking_session_id > 0
4	Check Top Wait Types	SELECT from dm_os_wait_stats ORDER BY wait_time_ms DESC
5	Find Expensive Queries	Activity Monitor Recent Expensive Queries OR dm_exec_query_stats
6	Check Index Fragmentation	dm_db_index_physical_stats — run if PAGEIOLATCH is the top wait
7	Check Available Memory	dm_os_sys_info — physical_memory_in_use_kb vs total_physical_memory_kb
8	Review SQL Server Error Log	Management → SQL Server Logs — recent errors, stack

		dumps, I/O warnings
--	--	---------------------

i

Document What You Find

Even if you resolve the problem, write down: what time the issue started, which session or query was the root cause, what you did to resolve it, and the outcome. This log becomes invaluable for pattern recognition in future incidents.